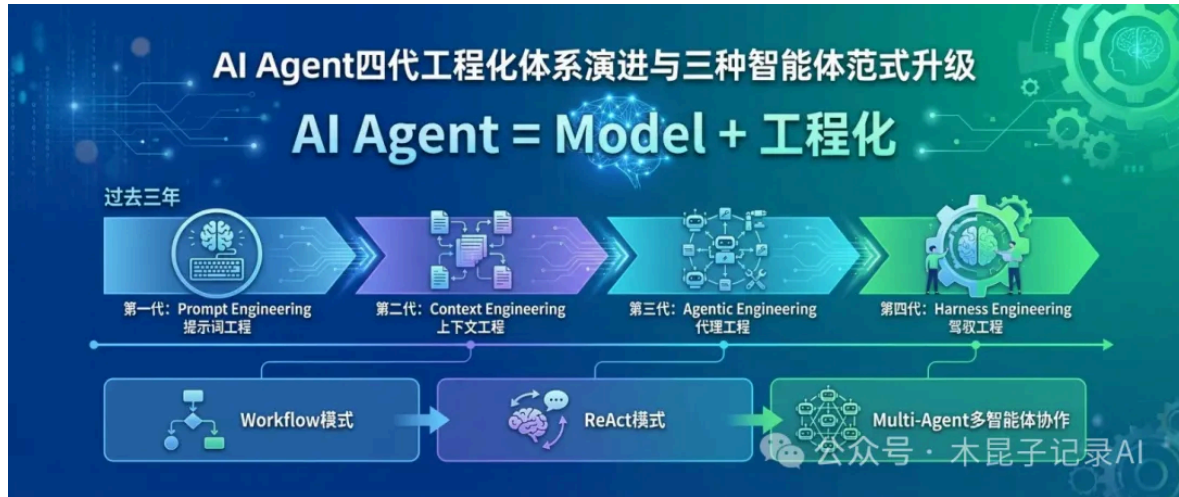


从Prompt到Harness：AI Agent四代工程化体系演进与三种智能体范式升级

原创 木昆子 木昆子记录AI 2026年3月30日 20:09 浙江

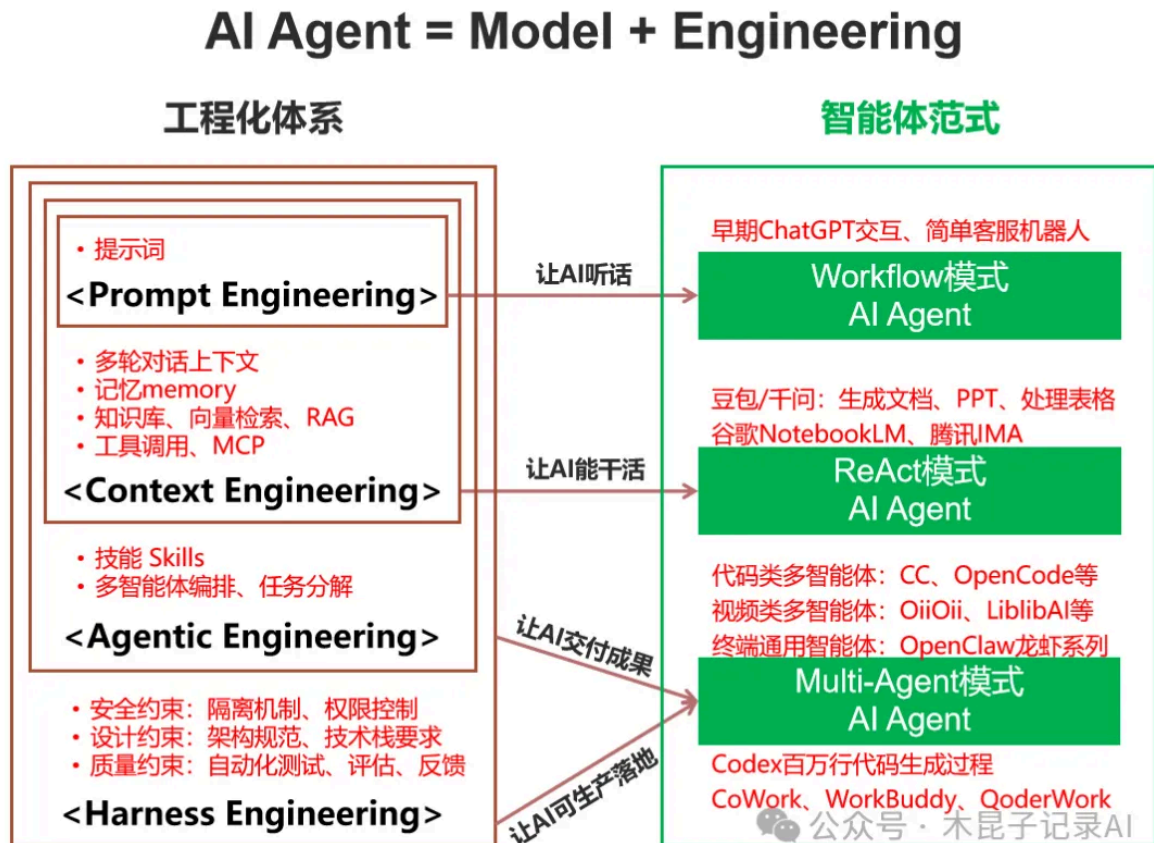
智能体AI Agent = Model + 工程化。



过去三年，AI工程化体系经历了从提示词工程（Prompt Engineering）到上下文工程（Context Engineering），再到 Agentic Engineering 和 驾驭工程（Harness Engineering）的四代跃迁。

AI智能体范式也相应从早期Workflow模式升级到ReAct模式，再升级到Multi-Agent多智能体协作模式。

梳理AI工程化体系和智能体范式之间的关系如下图：



每一代工程化体系和智能体范式的升级，对应着智能体从“能答”、“能用”到“好用”再到“可生产落地”的能力质变。

第一代：提示词工程 × Workflow 模式（2023–2024）

工程化体系：Prompt Engineering

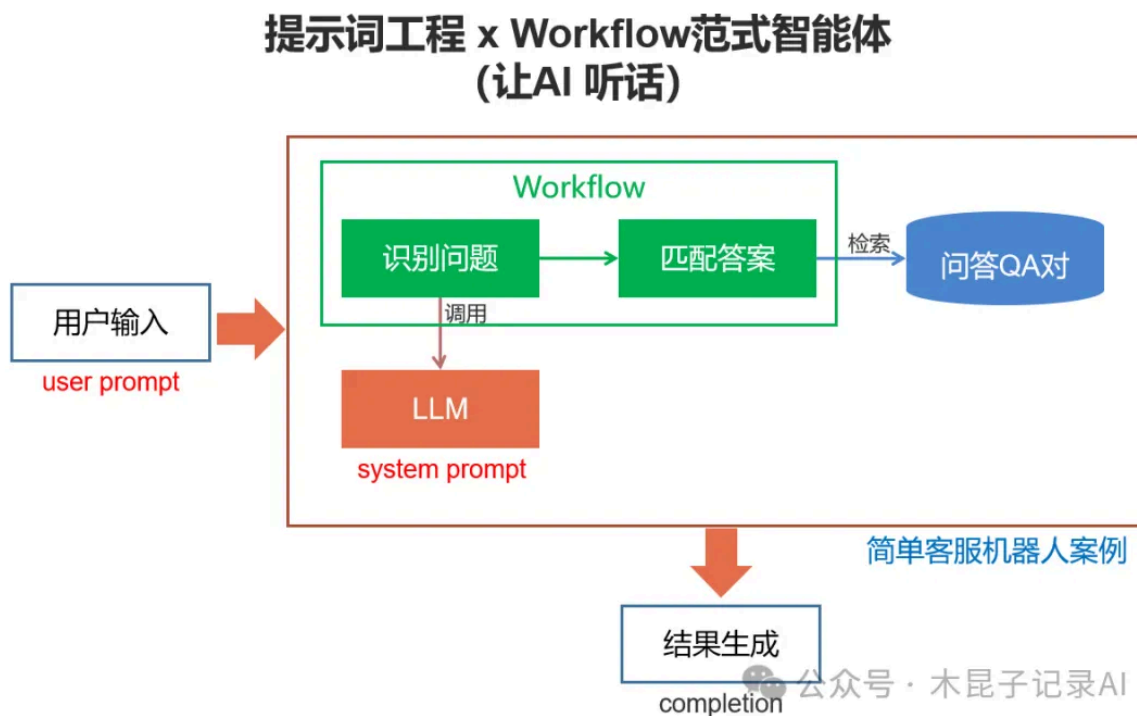
提示词工程的核心焦虑是“怎么把话说清楚”，所以通过反复调试指令措辞、格式、Few-shot 示例，让AI一次性给出好答案。

智能体范式：Workflow 模式

这一阶段的智能体遵循固定流程编排，像流水线一样按预设步骤执行：

- 输入结构化Prompt → LLM推理 → 输出结果
- 人工通过调整提示词措辞来干预输出质量

典型代表



- 早期ChatGPT交互：用户需要反复修改措辞才能得到理想答案。
- 固定流程智能体：基于LangChain或Dify等框架开发的确定性工作流Agent，如简单的客服机器人、单轮问答系统，参考本号案例《[从图片到答案，用Dify打造专属运维神器](#)》。

局限性

解决的是单次对话的质量问题，交互模式停留在“一问一答”，人和AI的关系像“出题者和答题者”。一旦任务复杂度提升或需要多轮协作，纯Prompt工程便显得力不从心。

第二代：上下文工程 × ReAct 模式（2025）

工程化体系：Context Engineering

上下文工程的提出，是因为大家发现**光靠Prompt不够**，AI需要看到相关文档、代码片段、历史对话、工具调用结果才能给出好答案。Shopify CEO Tobi Lutke将这一概念推至风口浪尖，AI 大神 Andrej Karpathy 进一步对其进行推广，获得业界广泛认可。

正如 Karpathy 所说“上下文工程是一门微妙的艺术与科学，旨在填入恰到好处的信息，为下一步推理做准备。说它是科学，是因为需要综合运用任务描述、少样本示例、RAG、多模态数据、工具、状态与历史记录、信息压缩等一系列技术”。

体系范围扩展（在提示词工程基础上增加）：

- 多轮对话历史与短期记忆
- 长期记忆（知识库、向量检索）
- MCP（Model Context Protocol）工具调用
- RAG检索增强生成

智能体范式：ReAct模式（Reasoning + Acting）

ReAct模式实现了“**思考-行动-观察**”的循环（参考本号文章《[从workflow到ReAct提升AI Agent智能化水平](#)》）：

1. **推理（Reasoning）**：LLM分析当前状态和目标
2. **行动（Acting）**：调用工具或生成响应
3. **观察（Observation）**：将工具返回结果纳入上下文
4. 循环直至任务完成

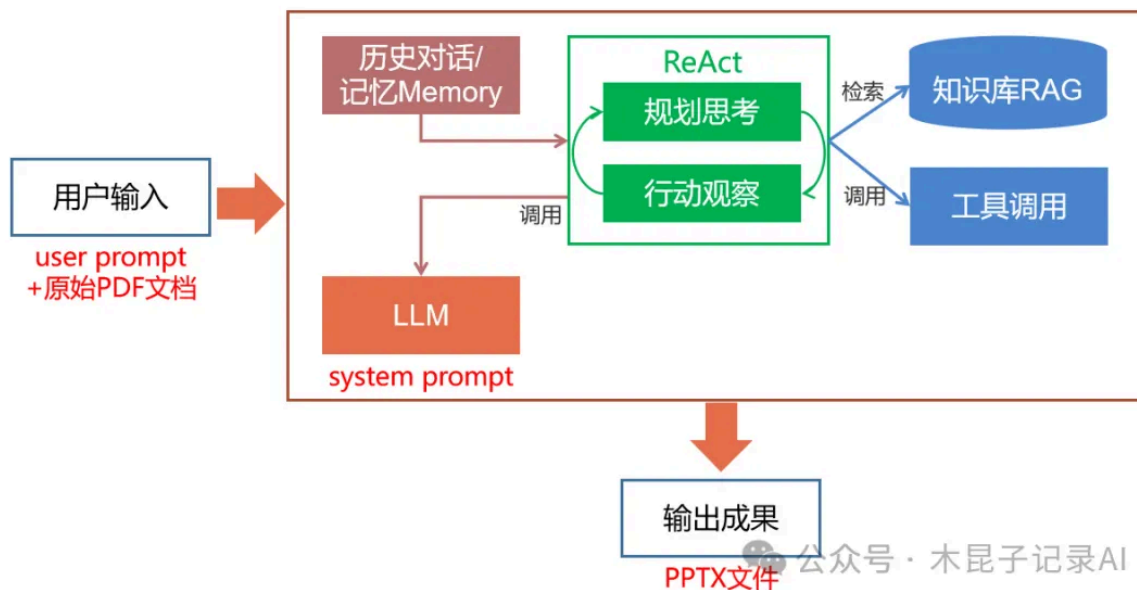
比如当你问“帮我查一下明天北京到上海的航班，选一个上午出发、价格低于1000元的，然后帮我订一张”时，它会：

思考：需要查询航班信息 → 行动：调用航班查询工具 → 观察：获取航班列表 → 思考：筛选符合条件（明天上午、<1000元）的航班 → 行动：调用订票工具 → 观察：订票成功 → 输出结果

这个过程完全在上下文管理下自动完成。

典型代表

上下文工程 x ReAct范式智能体 (让AI 能干活)



比如豆包、千问、Kimi等APP，已经不再只是“聊天机器人”，它们可以按需调用工具：

- 上传一份PDF，要求“生成一份总结PPT”，AI会调用工具读取并解析原始文档内容，规划PPT结构，调用生成工具，输出可直接使用的PPT文件，参考本号文章《[将文章转为演示文稿：AI生成PPT工具全面对比](#)》。
- 上传Excel表格，要求“分析销售数据，找出增长最快的三个品类，并生成图表”，AI会理解数据结构，进行数据分析，调用图表生成功能，交付完整报告，参考本号合集案例《[AI 数据分析](#)》。

个人知识库类产品更是上下文工程的典型代表（参考本号文章《[产品模式思考：深度拆解腾讯IMA和谷歌NotebookLM的三层知识架构](#)》）：

- 谷歌NotebookLM：上传你的笔记、文档、链接，AI基于这些“私有上下文”回答问题、生成摘要、甚至创建播客式对话（参考本号文章《[智能笔记新标杆：Google NotebookLM深度测评与实战技巧分享](#)》）。
- 腾讯IMA：个人知识管理工具，AI能基于你的知识库提供精准答案（参考本号文章《[产品模式思考：腾讯IMA知识库为什么成功？](#)》）。

第三四代：智能化工程 + 驾驭工程 × Multi-Agent模式（2025年底-）

工程化体系：Agentic Engineering

Andrej Karpathy后来提出Agentic Engineering智能化工程概念，聚焦“让AI能交付、能协作”，指导vibe coding（氛围编程）逐渐向可靠交付转变。所谓‘Agentic’，是因为99%的时间里你不再直接写代码，而是在编排智能体（Agents）完成工作，并充当监督者。而‘Engineering’则是为了强调这其中包含着艺术、科学与专业知识，是一项可以学习并不断精进的技能。

智能化工程包含编排多智能体、工具调用、任务分解等，人类负责设定目标、约束和质量标准，包括设计架构、定义边界、监督执行等。

体系范围扩展（在上下文工程基础上增加）：

- 技能（Skills）：可复用的专业技能模块
- 多智能体编排（Multi-Agent Orchestration）：协调多个专业Agent协作
- 风险控制：内建品质关卡、自动化测试、审计轨迹

工程化体系：Harness Engineering

前面Agentic Engineering虽然也包括风险控制、监督执行等，但更多还是侧重技能提炼、复用、多智能体编排等能力提升，使得AI从能干活往能真正交付生产可用成果迈进。

而近期HashiCorp 联合创始人 Mitchell Hashimoto 提出的驾驭工程（Harness Engineering），之所以得到业界更广泛的认可，则是在智能体已经足够强大的基础上，更加强调整要构建一整套系统来约束、引导和验证AI Agent的自主行为，让AI安全可靠地在生产环境中落地。

体系范围扩展（在智能化工程基础上更加强调）：

- 安全约束：权限控制、资源隔离等
- 设计约束：架构规范、技术标准等
- 质量约束：自动化测试、评估Eval、持续反馈等

驾驭工程核心设计哲学："每当你发现Agent犯了一个错误，你就花时间设计一个解决方案，使Agent永远不再犯同样的错误"。这不是单次优化，而是一套可积累、可进化、能持续收敛错误的闭环体系。

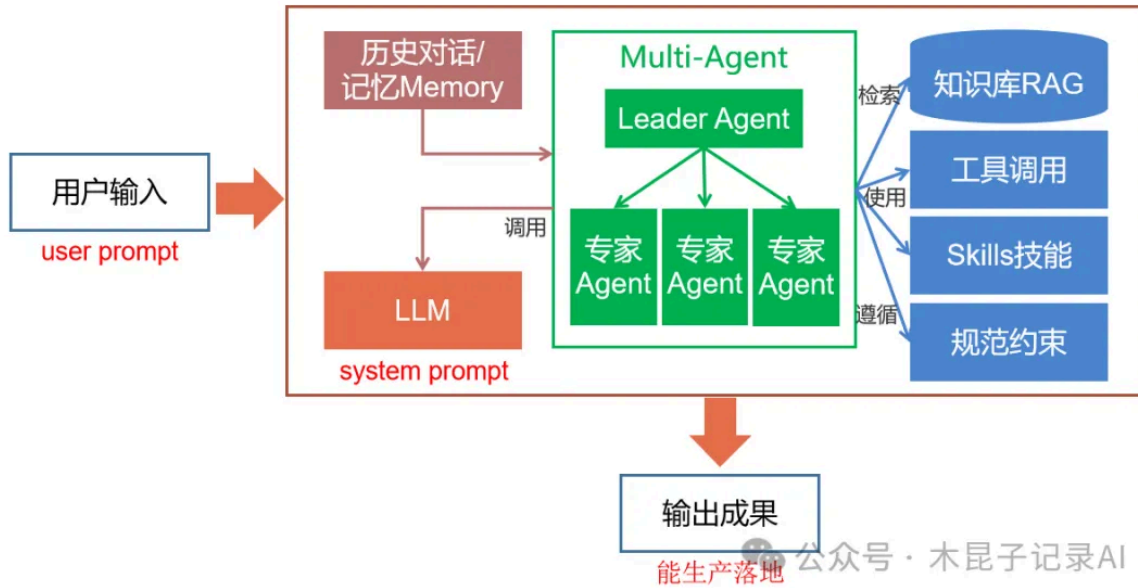
智能体范式：Multi-Agent协作模式

无论是Agentic Engineering还是Harness Engineering的工程化，主要还是依赖AI Agent从"单兵作战"进化为"团队协作"Multi-Agent：

- 星形拓扑架构：Leader Agent负责规划与指挥，Expert Agents并行执行
- 直接通信机制：Agent之间可直接对话，无需通过用户中转
- 上下文隔离：每个Agent拥有独立上下文空间，避免信息干扰
- 共享任务看板：实时同步任务状态与依赖关系

典型代表

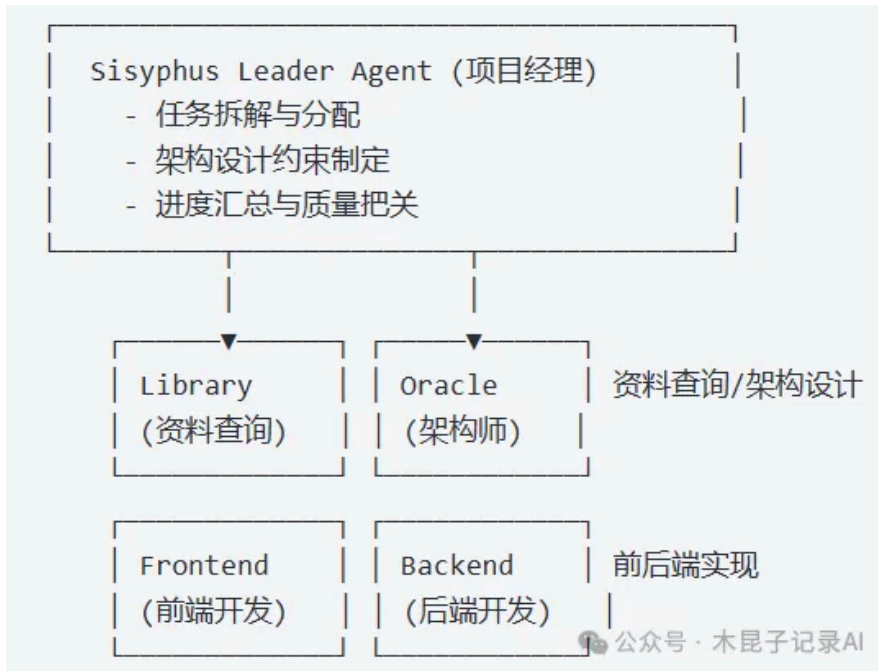
智能化工程+驾驭工程 x Multi-Agent模式 (让AI 能交付可生产落地的成果)



1. Coding Agent类产品：

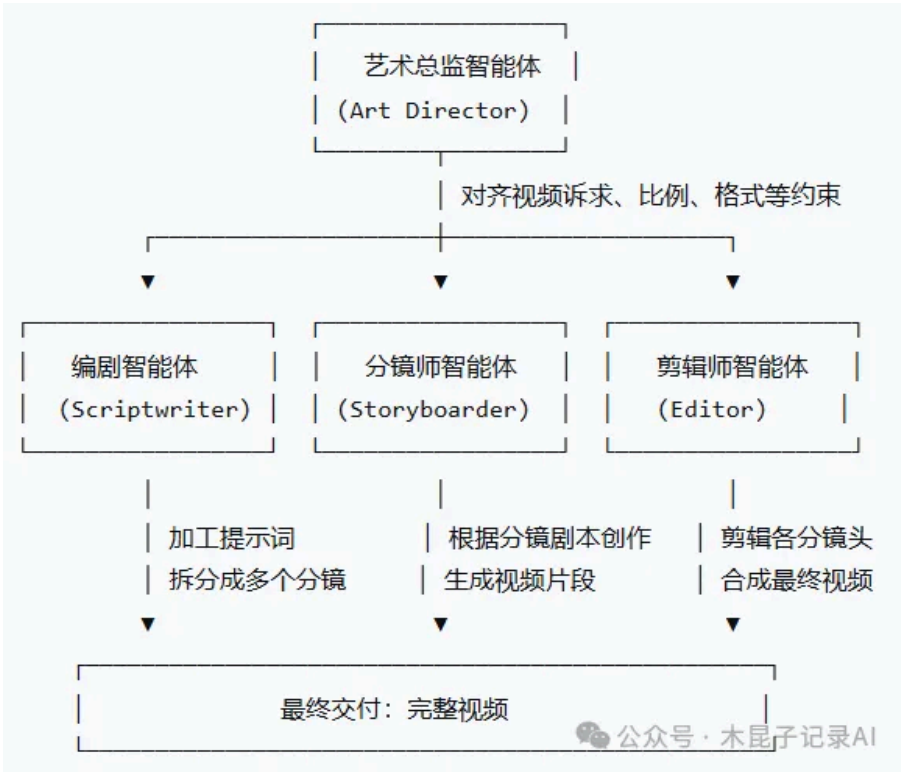
Anthropic 2025年推出的Claude Code通过Skills + Sub Agent机制，拉开了序幕，众多产品在此基础上，逐步完善多智能体协作机制。CC后续也推出的Agent Teams功能，实现向群体智能的演进。

以OpenCode+oh-my-opencode插件为例，看看OMO多智能体协作架构示意如下，核心就是Sisyphus西西弗斯的任务拆解与分配：



2. 视频生成多智能体：比如OiiOii

领先的视频生成平台已不再是根据提示词一次性输出5秒视频，而是典型的多智能体协作流程：



整个过程中，各环节智能体都在艺术总监设定的约束框架内工作，由艺术总监负责任务拆解与智能体分配，确保输出质量可控。

3. 终端个人通用智能体：OpenClaw（龙虾）系列演化

春节前后大热的OpenClaw代表了个人终端智能体的方向（参考本号合集[《龙虾OpenClaw部署》](#)）。其特点包括：

- 终端部署能力
- 访问终端文件、执行终端脚本、调用终端上的工具
- 可扩展的Skills生态
- 能对接各种IM渠道，方便手机远程操控

基于这些特点OpenClaw确实带来巨大的飞跃，它可以自主完成复杂任务并交付可用成果，充分利用终端能力，但也面临"终端失控、数据泄露"等安全挑战，需要Harness Engineering的约束体系。

所以类似腾讯WorkBuddy和阿里QoderWork等产品，则是在继承OpenClaw理念的基础上，进一步完善安全审计、权限管控、沙箱隔离等机制，参考本号文章[《腾讯WorkBuddy：OpenClaw 的合格平替（零门槛 + 不花钱 + 低风险）》](#)。

这正是第四代驾驭工程要解决的问题，让AI在生产环境中可靠运行，恰好对应了我们说的智能化工程（OpenClaw）与驾驭工程（WorkBuddy）的分野。

整体逻辑总结：从"适配模型"到"驾驭模型"

AI Agent的工程化演进完全贴合大模型行业的发展阶段：

1. Prompt Engineering（适配模型）：通过优化提示词适配大模型的生成逻辑，让AI听话按指令输出。

- 2. **Context Engineering（赋能模型）**：注入准确信息打破大模型知识边界，支持工具调用，让AI好用能干活。
- 3. **Agentic Engineering（增强模型）**：通过技能、编排与协作增强大模型，让AI能交付可用成果，完成长程任务。
- 4. **Harness Engineering（驾驭模型）**：通过全链路管控约束不确定性，让AI能规模化生产落地。

四者关系：不是替代，而是深度融合、互为支撑，Prompt是基础，Context是原材料，Agentic和Harness是生产框架。

结语

AI Agent的工程化演进揭示了一个核心规律：**大模型的能力边界不仅取决于模型本身，更取决于我们如何用工程化手段"驾驭"它**。从精心设计的提示词，到丰富的上下文供给，再到完整的约束与编排体系，每一代工程化都在回答同一个问题——**如何让AI在自主性与可控性之间找到最佳平衡点**。

当Multi-Agent系统能够像专业团队一样协作，当Harness Engineering能够确保长程任务的稳定交付，AI Agent才真正从"玩具"进化为"工具"，从"实验室"走向"生产线"。

本系列说明：基于AI工程化落地经验，结合对典型AI产品的使用体验，针对AI Agent的工程化体系演进与智能体范式升级进行梳理分析。

—End—

如果您觉得这篇文章对您有帮助欢迎**点赞**、**❤️**、**转发**三连击

也恳请您关注以下公众号+星标，这里有更多精彩思考和总结

您的支持是我继续写下去的动力💪



木昆子记录AI 🌟

专注AI项目落地中工程化实践及AI产品使用总结分享。

78篇原创内容



公众号

注：原创不易，合作请在公众号后台留言，未经许可，不得随意修改及盗用原文。

AI基础 · 目录 ≡

← 上一篇 · Streamable HTTP流式传输在LLM+MCP场景中的应用

